

Bayesian Generative Modeling of Handwritten Digits via Bernoulli Mixture Models

Akshay Daftardar

1 Introduction

Generative modeling, the task of learning a data distribution and sampling new observations from the distribution, is an area of active research within Machine Learning. Current approaches such as diffusion models and variational autoencoders (VAEs) can achieve remarkable image generation quality, but require large datasets, millions of parameters, and significant computational overhead to generate quality images. These models also function as black boxes. The learned representations are difficult to interpret, and uncertainty in the generative process is not directly quantified.

Bayesian inference offers a different approach. Rather than learning estimates of model parameters, a Bayesian generative model maintains posterior distributions, enabling uncertainty quantification and interpretable parameter estimates. Bayesian methods can operate effectively with small datasets, where the selection of a prior plays a meaningful role in the inference process.

This project explores Bayesian generative modeling through a Bernoulli mixture model applied to binarized handwritten digit images. Using only 25 images from the scikit-learn digits dataset, we perform posterior inference via Markov chain Monte Carlo (MCMC), analyze the learned class structure, and generate new digit images by sampling from the posterior predictive distribution. We also examine the role of prior selection and evaluate generation quality through checks on the posterior.

2 Data

The data contains 1,797 handwritten digit images from the scikit-learn digits dataset [1]. Each image is an 8×8 grayscale grid of pixel intensities ranging from 0 to 16. There are 10 total numbers, ranging from 0 to 9. Figure 1 shows the raw image data. To simplify the likelihood to a Bernoulli distribution,

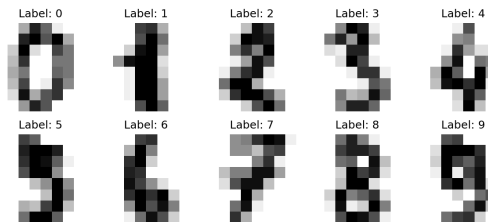


Figure 1: Raw Digits from scikit-learn dataset

```
Class distribution in sample:
Digit 0: 2 images
Digit 1: 1 images
Digit 2: 2 images
Digit 3: 2 images
Digit 4: 3 images
Digit 5: 4 images
Digit 6: 2 images
Digit 7: 1 images
Digit 8: 3 images
Digit 9: 5 images
Dataset: 25 images, 64 pixels each
Probability of Pixel being on: 0.37375
```

Figure 2: Class Distribution of binarized images

all pixel values were binarized using a threshold of 4. Pixels above this threshold were set to 1 (on) and all others to 0 (off). From the full dataset, 25 images were randomly subsampled using a fixed seed for reproducibility. The resulting class distribution is shown in Figure 2. Figure 3 shows a selection of the binarized images.

3 Model

The model assumes that each image is generated by:

1. Selecting a latent class k from mixing weights, where mixing weights are defined as the probabilities of selecting any class out of the 10 available classes.
2. Generating each pixel independently from a Bernoulli distribution.

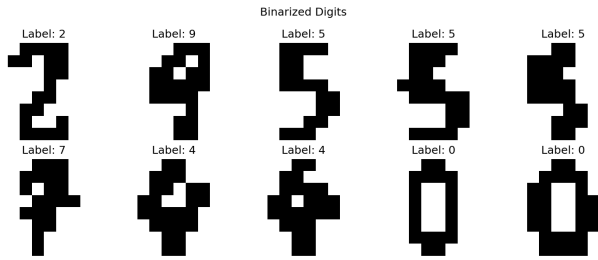


Figure 3: Binarized Images with Threshold > 4

Formally, we can say the model is the following:

$$\mathbf{w} \sim \text{Dirichlet}(\alpha \cdot \mathbf{1}_K) \quad (1)$$

$$\theta_{k,j} \sim \text{Beta}(a, b) \quad \forall k \in 1..K, j \in 1..D \quad (2)$$

$$x_i \sim \sum_{k=1}^K w_k \cdot \prod_{j=1}^D \text{Bernoulli}(x_{i,j} \mid \theta_{k,j}) \quad (3)$$

Where $K = 10$ is the number of components in our dataset, and D is the number of pixels in each image (64). The Dirichlet prior controls how evenly the model samples images across the given classes. The evenness of model sampling is controlled by the hyperparameter α . The parameters a and b of the Beta prior are determined through testing, and the Beta prior is set on all pixels. The Beta prior determines the probability of a given pixel to be on or off. The selection of these values will be discussed in more detail in Section 4. The Bernoulli distribution then determines the likelihood of the observed pixel values given the Beta Prior.

4 Prior Selection

Priors were selected for each of the major components of the model. This includes: 1. A Dirichlet distribution on the mixing weights. 2. A Beta Distribution on the pixels on/off probabilities θ .

We used the Dirichlet distribution here because of the uneven sampling of the data shown in Figure 2. Testing with different α values showed varying degrees of success. Initial testing was done with an α value of 0.5. This is a common starting point for the Dirichlet distribution but can lead to sparsity in the output if not all classes are required. This value led to mode collapse - the model concentrated most of its weight on a small subset of classes (Table 2 in Appendix A). This is further corroborated by the trace plots in Figure 4, which shows nearly

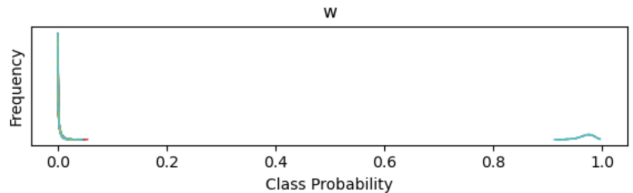


Figure 4: Frequency Plot for $\mathbf{w}$ with $\alpha = 0.5$

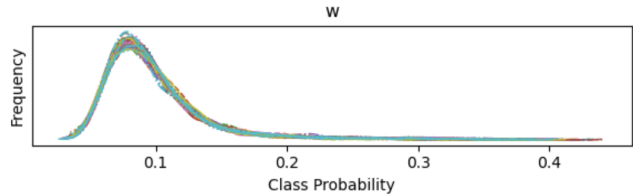


Figure 5: Frequency Plot for $\mathbf{w}$ with $\alpha = 25$

all probability mass concentrated on a few classes, with the remaining classes receiving close to zero weight. Testing with an α value of 10 led to similar results (Table 3 in Appendix A), although the weights were slightly more distributed than the α value of 0.5. Trace plots for this case were omitted for brevity. Based on the $\hat{r}$ values, it was clear that the weights were not converging, with some $\hat{r}$ values getting up to 1.73. Because we expected all 10 components to have representation in the data, we experimented with a large value of α of 25. This led to an expected result, where the mixing weights were roughly uniformly distributed but with a slight degree of variance (Table 4 in Appendix A). This is further corroborated by the trace plots in Figures 5 and 6. The trace plot shown in Figure 5 for $\mathbf{w}$ density concentrated near 0.1, which means that roughly all components will get sampled with similar probabilities. Figure 6 shows all ten class probabilities fluctuating in a band across 0.1. Unlike the Dirichlet(0.5) case, no one component dominates, demonstrating that this prior prevents mode collapse.

Next, we used a Beta Distribution to model the prior for the pixels on/ off probability θ . A value of 1 was chosen for a , and 2 was chosen for b . This

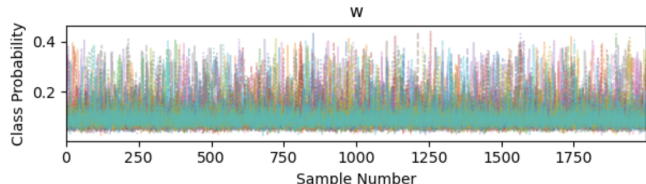


Figure 6: Trace Plot for $\mathbf{w}$ with $\alpha = 25$

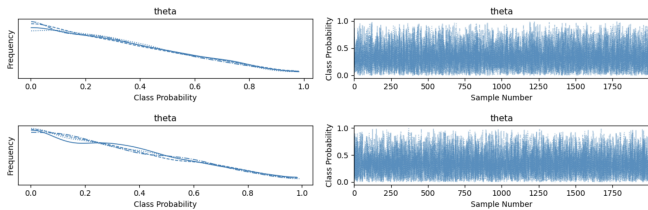


Figure 7: Trace Plots for θ under Dirichlet(0.5) and Beta(1, 2)

put the mean of the Beta distribution at $\frac{1}{3}$ (since $\mathbb{E}[\text{Beta}(a, b)] = \frac{a}{a+b} = \frac{1}{1+2} = \frac{1}{3}$). Referencing the data shown in Figure 2, the proportion of active pixels in the binarized dataset was approximately $\frac{1}{3}$, which makes $\text{Beta}(1, 2)$ a natural choice. The prior generally favors pixels being off, which is reflected by the fact that background pixels outnumber filled pixels in our sampled data.

The effect of mode collapse on θ is visible in the per-pixel trace plots. Figure 7 shows θ for a background pixel (pixel 0) and a fill pixel (pixel 28) under Dirichlet(0.5). Because the model collapsed to a few classes explaining all images, both pixels have diffuse posteriors. The model cannot distinguish background from fill. In contrast, Figure 8 shows the same pixels under Dirichlet(25). The background pixel is heavily concentrated near 0, while the stroke pixel peaks around 0.7, showing that the model has learned structure within each class.

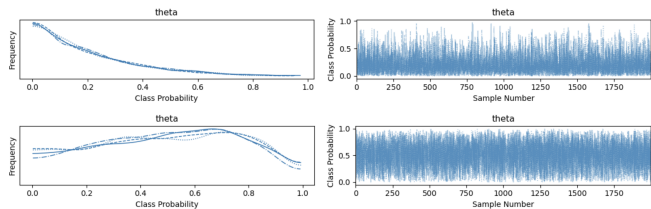


Figure 8: Trace Plots for θ under Dirichlet(25) and Beta(1, 2)

5 Likelihood Selection

The chosen likelihood function was Bernoulli. Because the data is binarized and the pixels can be either take a value of 1 or 0, a Bernoulli distribution became the natural choice. For each image, the likelihood is computed given the class specific pixel probabilities θ . Formally, this is described in Equation 3. The Beta prior and the Bernoulli likelihood

create a posterior pixel probability that remains in the Beta family due to the rules of conjugacy.

6 Inference

Posterior inference was performed using PyMC with the NUTS sampler. The model was run with 2,000 draws, 1,000 tuning steps, and a target acceptance rate of 0.98. A lower target acceptance rate (like 0.9) was resulting in divergence.

7 Results

The results in Table 1 shown post model fitting show interesting findings. The pointwise predictive density score favor the Dirichlet(0.5) model (deviance = 1313.7) over the Dirichlet(25) model (deviance = 1618.817). However, the lower-deviance model achieved this fit through mode collapse, where it concentrated most of its weight on a few classes. While this does result in a better fitting model, it defeats the purpose of a mixture model, which is supposed to identify distinct classes. The Dirichlet(25) model sacrifices a better fit in favor of a meaningful class structure. This problem presents a fundamental challenge with mixture models - the tradeoff is either to sacrifice fit or find meaningful class structure.

Table 1: Model comparison across Dirichlet prior settings

	rank	elpd_loo	p_loo	elpd_diff
Dirichlet 0.5	0	1313.694	39.531	0.000
Dirichlet 10	1	1425.372	27.507	111.678
Dirichlet 25	2	1618.817	20.129	305.123

7.1 Learned Class Prototypes

The learned class prototypes in Figures 9 and 10 show a heatmap of classes post model fit. As expected the heatmap in Figure 9 shows that Dirichlet(0.5) heavily favors a small subset of classes, while the heatmap in Figure 10 shows Dirichlet(25) assigns equal weight to all classes. Due to mode collapse in Dirichlet(0.5), the classes that have active pixels are not sharply differentiated. We would expect this result, because during mode collapse one class becomes the average of the dataset. However, the surprising

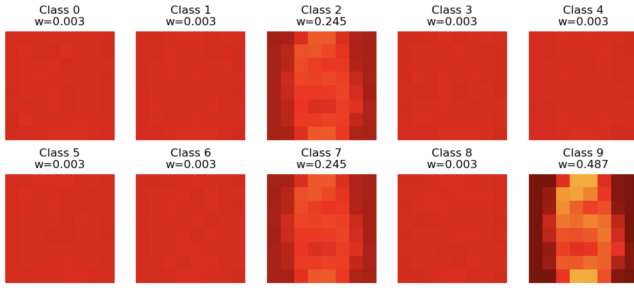


Figure 9: Learned Class Prototypes for $\mathbf{w}$ with Dirichlet(0.5)

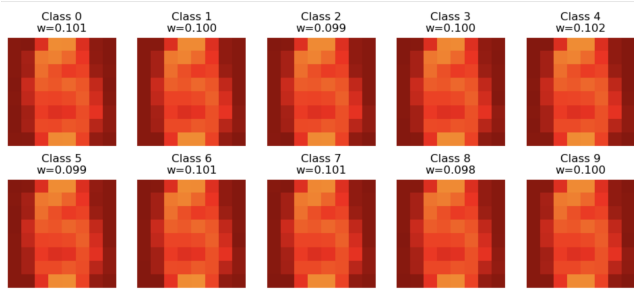


Figure 10: Learned Class Prototypes for $\mathbf{w}$ with Dirichlet(25)

result is in Dirichlet(25), where despite mode collapse not occurring, the classes are also not sharply differentiated. With only 25 images spread across 10 components, each class has approximately 2-3 images to learn from. This is too few data points for the posterior to move significantly from the prior, resulting in class prototypes that resemble slight variations of the overall data average.

7.2 Posterior Predictive Checks

Posterior predictive checks were performed using `pm.sample_posterior_predictive`. For each pixel position, we compared the mean value across all observed images to the mean value across all posterior predictive samples, using mean squared error (MSE) as the comparison metric. We then compared the real averages against the posterior predictive mean by plotting them on a scatter plot as seen in Figure 11. The Dirichlet(0.5) model had an MSE of 0.0017, and a scatter plot that showed a high degree of similarity with the real averages. Mode collapse creates a situation where the model is good at finding overall patterns in the images, thus resulting in a low mean squared error in the dataset. Meanwhile, the Dirichlet(25) model had an MSE of 0.04, while

the scatter plot in Figure 12 showed a lower degree of similarity. The posterior predictive checks for 12 also show the values compressed towards the mean of the prior of $\frac{1}{3}$. Pixels with real means near 0 or 1 were mapped to a more moderate posterior, showcasing the regularizing effects of the Beta(1,2) prior where each component only has 2-3 images to learn from.

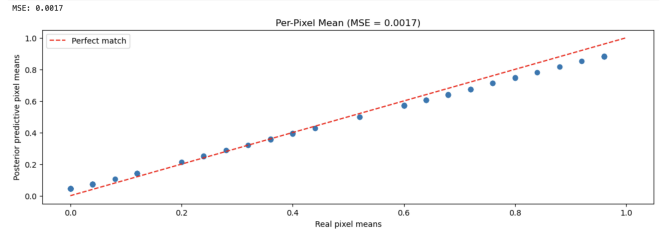


Figure 11: Real vs. Generated Per Pixel means for Dirichlet(0.5)

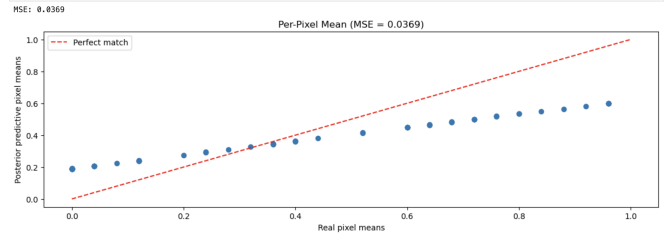


Figure 12: Real vs. Generated Per Pixel means for Dirichlet(25)

7.3 Generated Images

The images in Figures 13 and 14 reflect the Bayesian Generative Process. New images were generated by sampling from the posterior predictive distribution. For each generated image, a single posterior draw of $\mathbf{w}$ and θ was selected at random from the MCMC trace. A class was then randomly sampled from the mixing weights $\mathbf{w}$, and then each of the 64 pixels was independently sampled from a Bernoulli distribution. The Figures 13 and 14 show the generated images for the different α values of the Dirichlet distribution. Neither image shows coherent digits, which is likely due to the assumption that pixels are independent in the image, which ignores their continuous spatial structure.

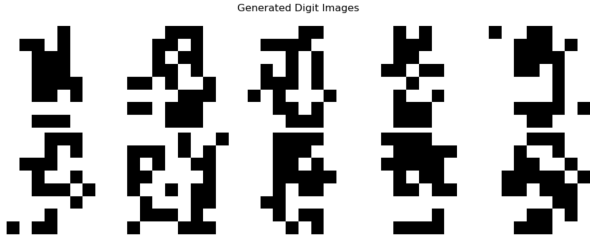


Figure 13: Generated Images for Dirichlet(0.5)

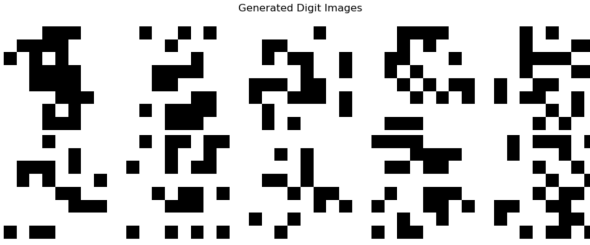


Figure 14: Generated Images for Dirichlet(25)

8 Discussion

While this venture into Generative Bayesian Modeling was an interesting exploration, certain assumptions were made that affected the quality of the output. The first assumption was that pixels are independent in an image. Present day Vision Transformers utilize attention between different patches of pixels in order to understand local and global spatial structure within images, while Convolutional Neural Networks utilize deep convolutional layers to understand local and global structure. These approaches sacrifice the interpretability and uncertainty quantification that is at the core of Bayesian Statistics. The second assumption was the binarization of pixel values, which reduced each pixel from a 0-16 range to a 0 or 1. This simplified the likelihood to a Bernoulli distribution but discarded information within the image that could have helped the model distinguish between classes.

Within the Bayesian Framework, we could have utilized hierarchical models to place a shared prior across pixels in the same spatial neighborhood, allowing the model to more accurately learn local spatial structure.

Finally, increasing the dataset size would have reduced the dependence on strong priors. The Dirichlet(25) prior was required to prevent mode collapse, but we could have picked a weaker prior and still achieved convergence with 100-200 samples.

9 Conclusion

This project explored Bayesian generative modeling through a Bernoulli mixture model applied to binarized handwritten digits. Using only 25 images, we performed full posterior inference via MCMC and examined how prior selection directly impacts model behavior. The results showed that weak priors led to mode collapse, while strong priors enabled meaningful class structure at the cost of aggregate fit. Posterior predictive checks confirmed that the model captured overall patterns, though the assumptions we made around conditional independence and binarization prevented generation of visually coherent digits. Despite these limitations, the project demonstrates that Bayesian mixture models provide an interpretable, uncertainty-aware framework for generative modeling.

References

- [1] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [2] Abril-Pla, O., et al. (2023). PyMC: A Modern, and Comprehensive Probabilistic Programming Framework in Python. *PeerJ Computer Science*, 9, e1516.
- [3] Kumar, R., Carroll, C., Hartikainen, A., & Martin, O. (2019). ArviZ a unified library for exploratory analysis of Bayesian models in Python. *Journal of Open Source Software*, 4(33), 1143.
- [4] Reding, A. (2024). ISYE 6420 – BUGS to PyMC. <https://areding.github.io/6420-pymc/>

A Supplementary Tables

Table 2: Posterior summary for mixing weights $\mathbf{w}$ with $\text{Dirichlet}(\alpha = 0.5)$.

	mean	sd	ess_bulk	r_hat
w[0]	0.003	0.005	7959.000	1.000
w[1]	0.003	0.005	7120.000	1.000
w[2]	0.245	0.419	7.000	1.530
w[3]	0.003	0.005	8316.000	1.000
w[4]	0.003	0.005	8216.000	1.000
w[5]	0.003	0.004	7895.000	1.000
w[6]	0.003	0.005	7371.000	1.000
w[7]	0.245	0.419	7.000	1.530
w[8]	0.003	0.005	7338.000	1.000
w[9]	0.487	0.484	6.000	1.730

Table 3: Posterior summary for mixing weights $\mathbf{w}$ with $\text{Dirichlet}(\alpha = 10)$.

	mean	sd	ess_bulk	r_hat
w[0]	0.038	0.013	5867.000	1.000
w[1]	0.193	0.270	7.000	1.530
w[2]	0.194	0.270	7.000	1.530
w[3]	0.037	0.013	5829.000	1.000
w[4]	0.038	0.013	6815.000	1.000
w[5]	0.038	0.013	6169.000	1.000
w[6]	0.038	0.013	5674.000	1.000
w[7]	0.349	0.312	6.000	1.730
w[8]	0.038	0.013	5519.000	1.000
w[9]	0.038	0.013	7443.000	1.000

Table 4: Posterior summary for mixing weights $\mathbf{w}$ with $\text{Dirichlet}(\alpha = 25)$.

	mean	sd	ess_bulk	r_hat
w[0]	0.101	0.047	2628.000	1.000
w[1]	0.100	0.047	2321.000	1.000
w[2]	0.099	0.043	2791.000	1.000
w[3]	0.100	0.044	2417.000	1.000
w[4]	0.102	0.048	2106.000	1.000
w[5]	0.099	0.043	2710.000	1.000
w[6]	0.101	0.047	2029.000	1.000
w[7]	0.101	0.046	2144.000	1.000
w[8]	0.098	0.041	2723.000	1.000
w[9]	0.100	0.046	2793.000	1.000